

HEALTH NORTH

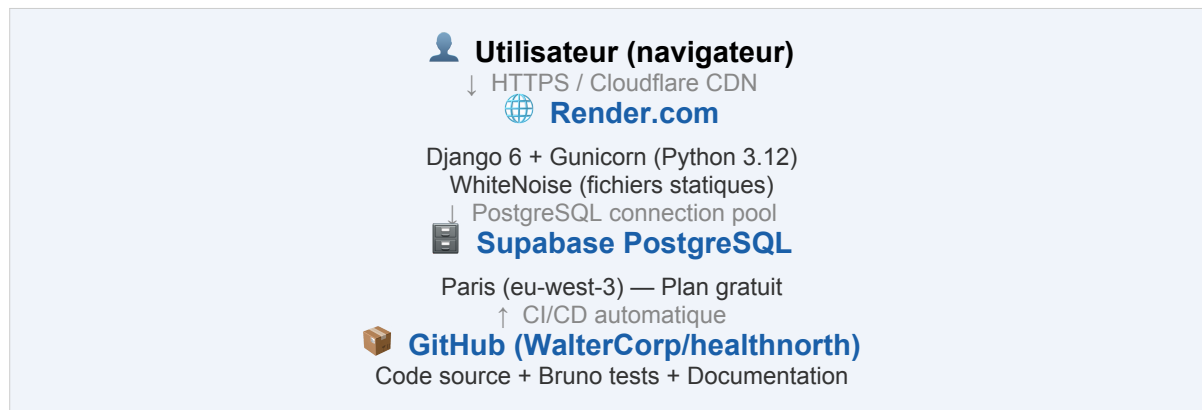
Infrastructure & Services

Dossier 2 — Missions 4 et 5

Projet	Health North — Application de réservation médicale
Dossier	Dossier 2 — Mise en place de l'infrastructure
Missions	Mission 4 — Gestion de tickets / Mission 5 — Authentification
Repository	https://github.com/WalterCorp/healthnorth
URL production	https://healthnorth.onrender.com

1. Architecture de déploiement

Health North est déployé sur une architecture cloud entièrement gérée, sans serveur à administrer. Voici le schéma d'architecture :



Composant	Technologie	Rôle
Serveur d'application	Render.com (Free tier)	Héberge Django via Gunicorn
Base de données	Supabase PostgreSQL	Stockage persistant des données
CDN / Proxy	Cloudflare	HTTPS, cache, protection DDoS
Fichiers statiques	WhiteNoise	Sert CSS/JS depuis Django
CI/CD	GitHub → Render	Déploiement automatique sur push main
Versionning	Git / GitHub	Code source et documentation

2. Mission 4 — Gestion de tickets

La gestion de tickets est un élément fondamental du suivi de projet professionnel. Elle permet de tracer les fonctionnalités développées, les bugs corrigés et les améliorations apportées, en associant chaque modification du code à un ticket identifié.

2.1 Outil choisi : GitHub Issues

GitHub Issues a été retenu comme outil de gestion de tickets pour les raisons suivantes :

- Intégration native avec le repository GitHub — pas d'outil supplémentaire à installer
- Chaque ticket est associé à un commit Git permettant la traçabilité entre code source et évolution fonctionnelle
- Interface simple et accessible — le jury peut consulter l'historique directement sur GitHub
- Permet de créer des labels, assigner des tickets et suivre leur statut (open/closed)
- Gratuit et sans limite pour les repositories publics

2.2 Tickets créés

10 tickets ont été créés et fermés sur le repository GitHub, couvrant l'ensemble des fonctionnalités développées :

#	Titre	Mission couverte
1	feat: implement user authentication (login/register)	Projet 2 — Mission 1
2	feat: implement profile and password update	Projet 2 — Mission 2
3	feat: implement appointment booking workflow	Projet 2 — Mission 3
4	feat: implement appointment list view	Projet 2 — Mission 4
5	feat: implement Django admin backend	Projet 2 — Mission 5
6	feat: implement JSON API endpoints for appointment form	Dossier 3 — Mission 2
7	fix: use region slugs in fixtures to match model choices	Correction de bug
8	test: add Bruno API test collection	Dossier 3 — Mission 3
9	style: redesign UI with blue theme and Bootstrap icons	Amélioration UX
10	feat: add appointment edit and documents pages	Projet 2 — Mission 3/4

2.3 Convention de nommage

Les tickets suivent la convention Conventional Commits :

- feat: — nouvelle fonctionnalité
- fix: — correction de bug
- style: — modification visuelle sans impact fonctionnel
- test: — ajout ou modification de tests

- docs: — documentation

2.4 Captures du repository GitHub

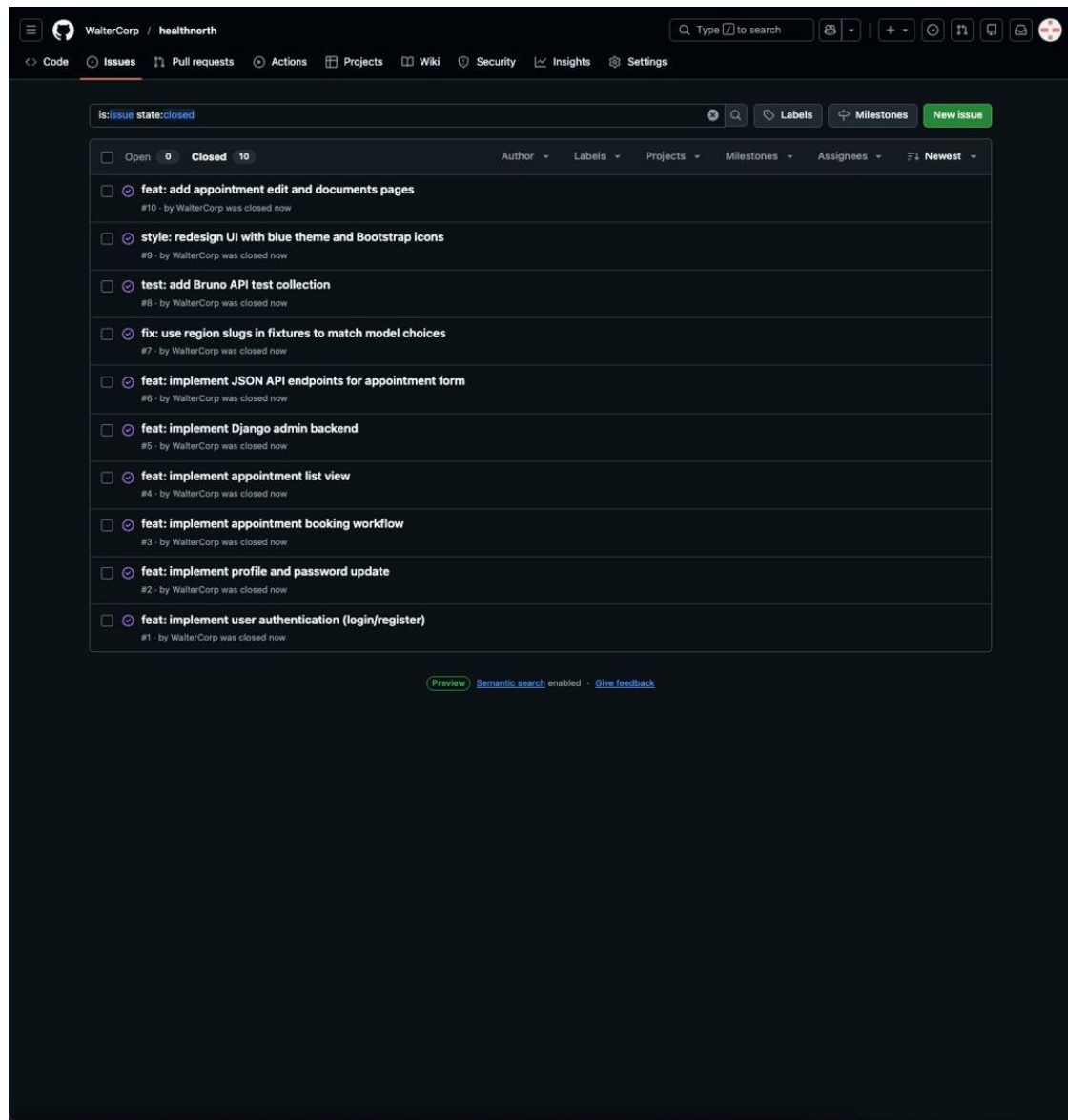


Figure 1 — GitHub Issues : 10 tickets fermés

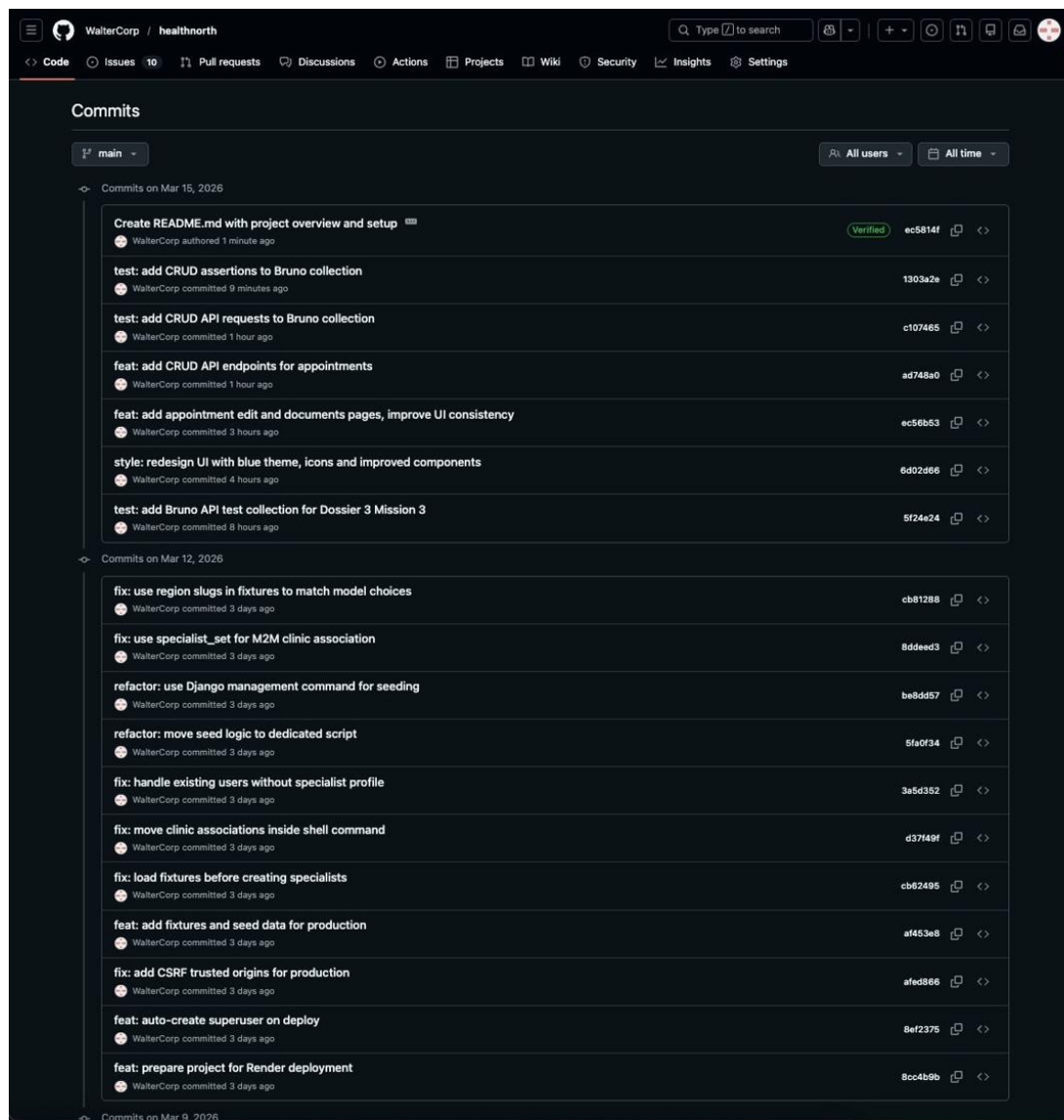


Figure 2 — Historique des commits GitHub

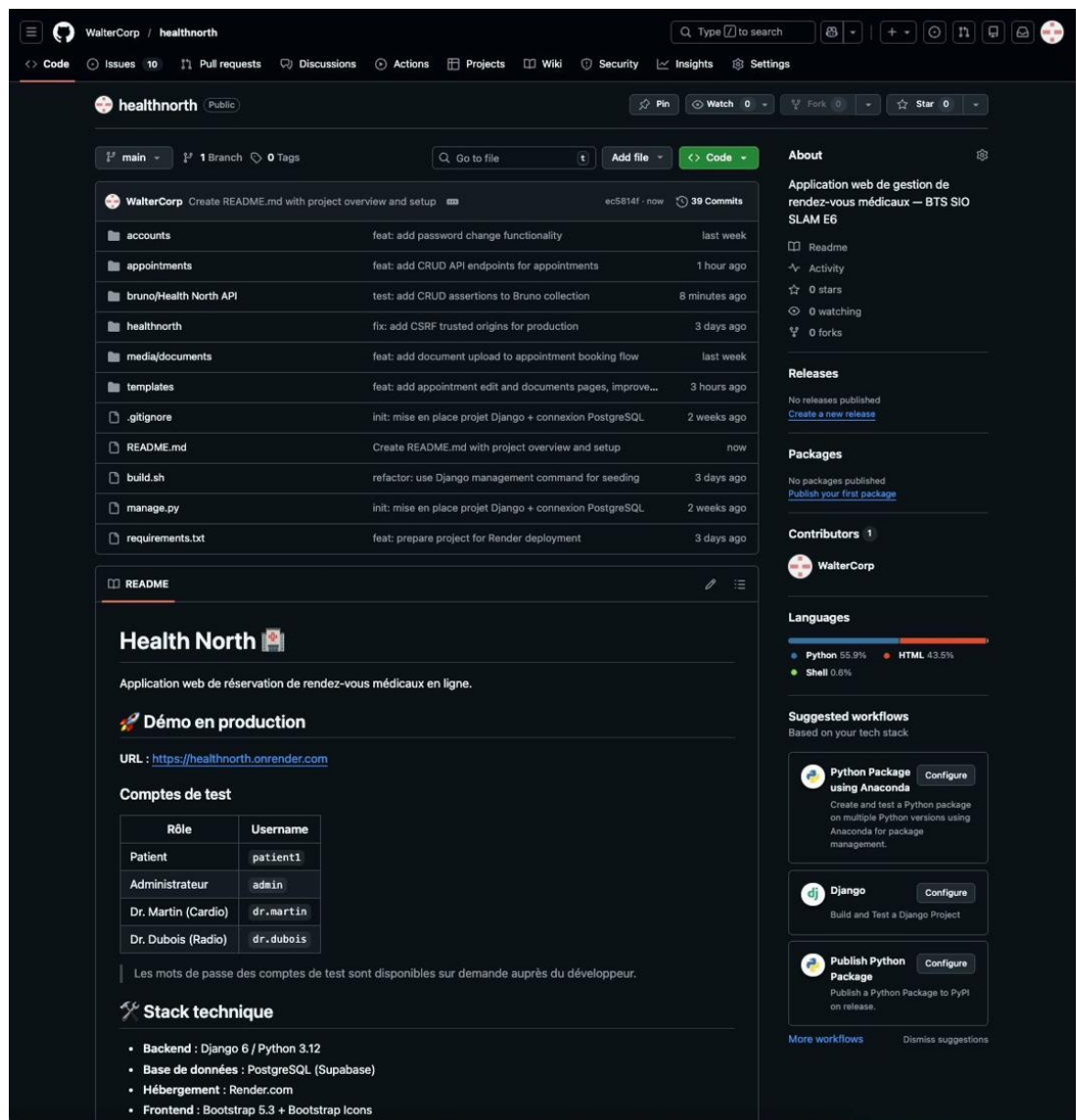


Figure 3 — Structure du repository GitHub

3. Mission 5 — Service d'authentification

L'énoncé demande la mise en place d'un service d'authentification (ex : openldap, active directory, etc.). Dans le cadre du projet Health North, le service d'authentification retenu est le système d'authentification natif de Django.

3.1 Justification du choix

- Django auth est conçu pour les applications web — il gère nativement les sessions HTTP, les cookies et la protection CSRF
- OpenLDAP et Active Directory sont des services d'annuaire adaptés aux réseaux d'entreprise internes, pas aux applications web publiques
- Django auth s'intègre directement avec la base de données PostgreSQL — pas de service externe à maintenir
- Le modèle User personnalisé (AbstractUser) permet d'étendre les attributs sans recréer toute la logique d'authentification

3.2 Fonctionnalités du service d'authentification

Fonctionnalité	Implémentation dans Health North
Hachage des mots de passe	PBKDF2-SHA256 — algorithme Django par défaut
Sessions utilisateur	Cookie sessionid géré par Django SessionMiddleware
Protection CSRF	Token CSRF sur tous les formulaires POST
Contrôle d'accès	Décorateur @login_required sur toutes les vues sensibles
Validation des mots de passe	4 validateurs Django : longueur, complexité, similarité, communs
Niveaux de droits	is_staff (admin), is_superuser, utilisateur standard
Isolation des données	Filtrage patient=request.user sur tous les RDV

3.3 Modèle utilisateur personnalisé

```
class User(AbstractUser):  
    phone = models.CharField(max_length=20, blank=True)  
    address = models.TextField(blank=True)  
    date_of_birth = models.DateField(null=True, blank=True)
```

Ce modèle est déclaré dans settings.py via AUTH_USER_MODEL = 'accounts.User'.

3.4 Flux d'authentification

1. Le patient soumet le formulaire de connexion (username + mot de passe)
2. Django vérifie les credentials via authenticate() — compare le mot de passe haché en base

- 3. En cas de succès, Django crée une session et pose le cookie sessionid dans le navigateur
- 4. À chaque requête, Django vérifie le sessionid et charge l'utilisateur correspondant
- 5. Le décorateur `@login_required` vérifie que l'utilisateur est authentifié avant d'autoriser l'accès
- 6. À la déconnexion, Django supprime la session côté serveur et efface le cookie

3.5 Sécurité implémentée — Réponse aux risques ISO 27001

Risque ISO 27001	Mesure implémentée
Mots de passe simples (3 car.)	Minimum 8 caractères + 4 validateurs Django
Utilisateurs non nominatifs	Chaque compte est nominatif (prénom + nom obligatoires)
API en accès libre	<code>@login_required</code> sur tous les endpoints API JSON et CRUD
Accès aux données d'autrui	Filtrage strict <code>patient=request.user</code> sur chaque vue
Vol de données de santé	Sessions sécurisées HTTPS via Render + Cloudflare